Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación

Trabajo Terminal No. ____ - ____

*Alumnos: *García Rodríguez Erick Daniel, Sánchez García Claudia Emilia*

Directores: Portillo Cedillo Manuel, Aragón García Maribel

**e-mail : e.danielgrz10@gmail.com*

Resumen – Este Trabajo Terminal propone el desarrollo de un sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación, como Fundamentos de programación y Algoritmos y Estructuras de Datos. Para cumplir con este objetivo se utilizará Comparación semántica basada en técnicas de Procesamiento de Lenguaje Natural (PLN) y representaciones vectoriales densas (embeddings) para contrastar las soluciones de los alumnos con el material proporcionado por el profesor. Además, se evaluará la similitud con código generado por modelos de Inteligencia Artificial (IA) para detectar similitudes significativas en el código.

Palabras clave – Aprendizaje automático, Inteligencia Artificial, Procesamiento de Lenguaje Natural, Similitud semántica de código.

1. Introducción

La validación de código en contextos académicos, especialmente en cursos introductorios de programación, representa un desafío fundamental para garantizar la integridad y la calidad del aprendizaje de los estudiantes [1]. Diversos estudios han demostrado que modelos como ChatGPT son capaces de resolver correctamente ejercicios de programación de nivel introductorio, lo que facilita que los estudiantes los utilicen para entregar soluciones automáticas sin comprender realmente el funcionamiento del código [2]. Esto genera un riesgo de aprendizaje superficial y de dependencia de la IA en lugar de la adquisición de competencias propias.

A nivel nacional, algunas universidades han comenzado a establecer lineamientos y reestructuraciones para el uso de IA en entornos educativos, con el fin de evitar plagio, fraude académico y pérdida de habilidades en los estudiantes de programación. [3]. Por ejemplo, algunas instituciones implementan políticas muy básicas como prohibir el uso del celular durante las clases para evitar que los estudiantes accedan a herramientas de IA. Sin embargo, estas medidas resultan limitadas y poco acordes con la realidad tecnológica actual, ya que no resuelven el problema de fondo, pues los alumnos pueden seguir utilizando la IA fuera del aula sin que los profesores puedan verificarlo [3]. Estos antecedentes refuerzan la necesidad de desarrollar sistemas de validación que consideren la similitud semántica de los programas y permitan una evaluación más justa en el contexto de cursos de programación.

Este desafío está presente en unidades de aprendizaje introductorias de programación en C/C++ impartidas en la Escuela Superior de Cómputo (ESCOM), como Fundamentos de programación y Algoritmos y Estructuras de Datos. En estas unidades de aprendizaje, los profesores se enfrentan a la dificultad de evaluar manualmente las soluciones de los alumnos. Este proceso consume tiempo y es propenso a errores, especialmente cuando existen múltiples soluciones funcionalmente equivalentes, pero sintácticamente diferentes.

Según lo descrito en [4] se han identificado 3 enfoques para comparar código:

Enfoque estructural: se centra en la organización y forma del código. Emplea representaciones como Árbol de Sintaxis Abstracta (AST) para capturar su estructura jerárquica. También se utilizan Grafos de Flujos de Datos (DFG) para representar dependencias entre variables [5] utilizando Distancia de Edición del Árbol de Sintaxis (TSED) se puede calcular la similitud estructural mediante las operaciones necesarias para transformar un AST en otro [4].

Modelos más actuales utilizan estas representaciones mediante Redes Neuronales Gráficas (GNN). Mientras que los más tradicionales que utilizaban comparación basada en cadenas, cuentan con muchas limitaciones como ignorar las características gramaticales y estructurales [5].

Enfoque funcional – Comprueba si dos códigos fuente logran el mismo resultado o comportamiento. La Coincidencia de Ejecución (Execution Match) es una métrica que comprueba la consistencia de los resultados de ejecución frente a una lista de resultados esperados. Se enfoca en la equivalencia de la funcionalidad practica [4].

Enfoque semántico – Busca comprender el significado subyacente del código más allá de su forma superficial o estructural. Mediante la creación de representaciones vectoriales densas (embeddings) de código o sus resúmenes se logra obtener su significado contextual [1], [6], [7]. Estos embeddings permiten hacer búsquedas de similitud basadas en el significado del código, no solo en las coincidencias léxicas . La Similitud Semántica es una medida utilizada para cuantificar que tan cercanos son dos textos (códigos en este caso) en términos de lo que realmente significan. A continuación, se mencionarán algunos de los modelos de aprendizaje automático más relevantes para la validación de código:

UniXcoder – Integra comentarios y AST, mostrando buen rendimiento en similitud de código en C/C++ y logra comparar la funcionalidad más allá de la estructura superficial [1].

CSSAM – Utiliza análisis semántico y estructural mediante AST y DFG, mejorando la detección de similitudes semánticas profundas [5].

CodeBERT / GraphCodeBERT – CodeBERT genera embeddings de código, mientras que GraphCodeBERT añade grafos de flujo de datos para capturar mejores relaciones estructurales [1], [6].

Universal Sentence Enconder (USE) / SenteceBERT – Son modelos que generan representaciones vectoriales densas de oraciones, capturando su significado contextual [8].

SimLLM – Enfoque novedoso basado en LLM para calcular similitud semántica en resúmenes de código. Utiliza preentrenamiento basado en permutaciones de entrada para entender la semántica concisa de los resúmenes de código, generando embeddings semánticos que tienen una alta correlación con las calificaciones humanas [7].

Software	Características principales	Limitaciones
Gradescope	Evaluación estructural y funcional básica muy utilizada en plataformas en línea .	No ofrece “crédito parcial” para sinónimos o equivalencia semánticas [9].
CodeGrade	Plataforma para detección de plagio [9].	Se puede evitar con ligeros cambios en el código [9].
MOSS (Measure Of Software Similarity)	Soporta más de 24 lenguajes, utilizando un algoritmo de huella digital de documentos (“Winnowing”) basado en hashing [1], [9].	Se puede manipular el código para evitar la detección ya que no detecta estructuras de código similares con diferencias superficiales [9].
Turnitin	Sistema de detección de plagio de texto, se puede aplicar a código fuente [9].	Limitado para plagio profundo de código, requiere suscripción [9].
Solución propuesta	Comparación semántica de código mediante embeddings, detecta equivalencias funcionales y código generado por IA. Con enfoque en cursos introductorios de C/C++.	Cuenta con limitaciones para implementaciones más complejas de C/C++.

Tabla 1. Resumen de productos similares

2. Objetivo general

Desarrollar una propuesta de sistema para la validación automática de código en unidades de aprendizaje introductorias de programación en C/C++, mediante la detección de similitud semántica entre programas del profesor, respuestas de los estudiantes, y código generado por IA, con el fin de identificar patrones de similitud, ayudar a detectar posibles plagios, reducir el esfuerzo de la evaluación manual y fomentar la integridad académica, favoreciendo un aprendizaje justo para los estudiantes.

Objetivos específicos

1.- Investigar y comparar los enfoques existentes de detección de similitud de código (estructural, funcional y semántico), identificando sus ventajas y limitaciones en el contexto académico de cursos introductorios de programación.

2. Identificar las características semánticas relevantes en programas escritos en C/C++, que permitan comparar soluciones equivalentes, aunque sintácticamente diferentes.

2.- Seleccionar y adaptar modelos de representación semántica y aprendizaje automático (por ejemplo, CodeBERT, GraphCodeBERT, UniXcoder o embeddings textuales) adecuados para el análisis de código en C/C++.

3.- Diseñar e implementar un sistema de comparación de código que integre técnicas de similitud semántica, capaz de identificar equivalencias funcionales aun cuando el código difiera en su forma superficial.

5.- Probar la herramienta con casos reales de estudiantes de cursos introductorios de programación en la ESCOM, analizando su efectividad y utilidad en el proceso de evaluación académica.

3. Justificación

Los profesores de unidades de aprendizaje introductorias en C/C++ de ESCOM enfrentan el desafío de evaluar manualmente una gran cantidad de tareas. Este proceso toma mucho tiempo para detectar si el estudiante entendió realmente el problema y logró implementar una solución valida al problema que utilice lo enseñado en el curso o si replicó el código de una IA generadora sin comprender realmente lo que este hace.

El presente trabajo terminal, propone el desarrollo de un sistema de comparación de código mediante similitud semántica para comparar programas de alumnos contra soluciones esperadas por el profesor, detectando equivalencias funcionales y posibles plagios.

Esta propuesta aporta originalidad al utilizar representaciones semánticas de código y adaptarlas al contexto educativo local. A comparación de sistemas existentes, que su enfoque esta en comparar las coincidencias textuales o estructurales, la solución propuesta por este Trabajo Terminal busca comprender el significado del código y su funcionalidad, logrando una evaluación justa y representativa.

La propuesta también es un gran reto por su complejidad en el campo de la Inteligencia Artificial, ya que implica generar y manipular representaciones semánticas de código, aplicando reconocimiento de patrones y aprendizaje automático para determinar la similitud entre códigos.

Los principales beneficiaros serán los profesores de las unidades de aprendizaje introductorias a la programación en C/C++ como Fundamentos de programación y Algoritmos y Estructuras de Datos, los cuales podrán reducir su carga de evaluación manual.

Las herramientas actuales como MOSS o Turnitin tienen limitaciones en cuanto a la comparación y detección de plagio, ya que el resultado de estas puede variar con cambios superficiales al código, en comparación, la solución propuesta no solo mide similitud superficial de código, sino que genera un índice de similitud semántica que permite determinar si dos soluciones son funcionalmente equivalentes, incluso si están escritas de manera diferente.

El desarrollo del sistema propuesto es viable considerando el tiempo disponible y el alcance del Trabajo Terminal. Se plantea que, al combinar y adaptar recomendaciones y conclusiones de trabajos de investigación previos sobre validación automática de código y análisis semántico, es posible aplicar estas estrategias al contexto de los cursos introductorios de programación en C/C++ en ESCOM. Con este enfoque se puede llevar la propuesta a un producto coherente y funcional para el entorno académico que se plantea, logrando cumplir los objetivos de evaluación, realizando la comparación de soluciones y apoyando a la integridad académica.

4. Productos o Resultados esperados

La Figura 1 muestra a grandes rasgos la arquitectura general del sistema propuesto. Se puede observar que el sistema recibe las siguientes entradas: El código del profesor, las soluciones de los estudiantes y, opcionalmente código generado por herramientas de inteligencia artificial. Posteriormente se realizará la comparación semántica para determinar el grado de similitud entre los códigos. Por último, el sistema generara un reporte indicando el porcentaje de similitud y de plagio.

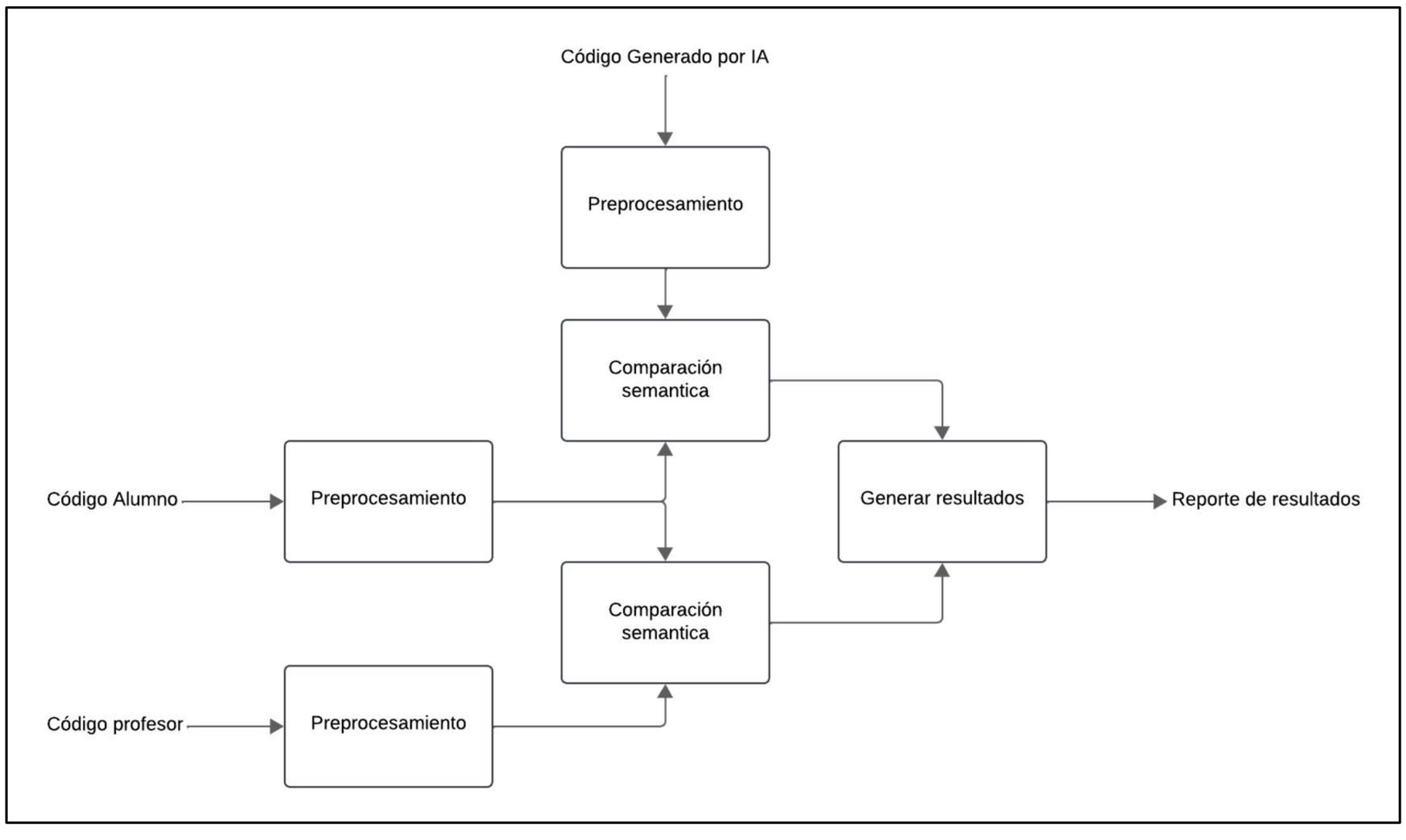

Figura 1. Arquitectura del sistema.

A continuación, se muestra una lista de los productos esperados del TT:

1. Aplicación del sistema.
2. Documentación técnica del sistema.
3. Manual de usuario.
4. Reporte técnico del sistema.

5. Metodología

El desarrollo se llevará a cabo bajo la metodología Komorebi, siguiendo un enfoque ágil e iterativo que se divide en cuatro fases principales: Definición, Ingeniería de Datos, Modelización y Despliegue. Cada una de estas fases tienen subetapas específicas que aseguran la calidad del sistema, permitiendo su desarrollo y mejora continua.

1. **Definición.**
   Fase inicial donde se establecen las estrategias y técnicas al usar en los proyectos.
   a. Planificación:
      Se establecerán los requisitos técnicos y recursos necesarios. Se evaluará la viabilidad técnica y de negocio, identificando factores críticos de éxito y se realizará un análisis preliminar de riesgos. Además, se definirán métricas de éxito que permitirán medir el progreso y resultados del proyecto.

2. **Ingeniería de Datos.**
   Fase donde se adquieren, procesan y preparan los datos para alimentar al modelo.
   a. Adquisición de datos:
      Se recolectarán la información de entrada necesarios para el sistema, estableciendo características que los datos deben cumplir.

   b. Preparación de datos:
      Se analizan los datos para comprenderlos, y de ser necesario realizar procesos de limpieza, normalización, aumentado, exploración y visualización de datos.

3. **Modelización.**
   Se selecciona y construye el modelo mas adecuado de aprendizaje automático para posteriormente evaluarlo.
   a. Entrenamiento de modelos:

Se probarán distintos algoritmos o arquitecturas para seleccionar el mas adecuado para la comparación semántica y se ajustarán sus parámetros. Para la generalización del modelo, se utilizarán técnicas como validación cruzada.

b. Evaluación de modelos:
Se utilizarán métricas como precisión, recall, F1-score para evaluar los modelos, considerando la robustez del modelo frente a diversos escenarios, así como aspectos de explicabilidad para comprender mejor las decisiones tomadas por el modelo.

4. **Despliegue.**
Se lleva al modelo entrenado a un entorno de uso real para validar su utilidad práctica.
   a. Despliegue de modelos:
   El modelo se integrará en la aplicación final, aplicando técnicas de optimización para reducir tiempos de ejecución y consumo de recursos.

   b. Monitorización de modelos:
   Se evaluará el desempeño del modelo en condiciones reales con las métricas definidas y se realizará mantenimiento iterativo para asegurar el correcto funcionamiento del modelo y el sistema.

Se utilizarán las siguientes herramientas y técnicas para el desarrollo del Trabajo Terminal:

1. Técnicas de representación estructural y semántica de código: Como Arboles de Sintaxis Abstracta (AST), Grafos de Flujos de Datos (DFG) y representaciones vectoriales densas (embeddings), seleccionando las técnicas mas adecuadas para abordar el problema.
2. Métricas de similitud.
3. Modelos de aprendizaje automático y aprendizaje profundo.
4. Lenguajes de programación: Python.
5. Bibliotecas de Aprendizaje automático y Procesamiento de Lenguaje Natural.
6. Sistema de control de versiones: Git.

6. Cronograma

CRONOGRAMA García Rodríguez Erick Daniel TT No:

Título del TT: Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación

Actividad	FEB	MAR	ABR	MAY	JUN	AGO	SEP	OCT	NOV	DIC	ENE
Planificación											
Adquisición de datos											
Preparación de datos											
Entrenamiento de modelos											
Evaluación de modelos											
Despliegue de modelos											
Monitorización de modelos											
Documentación técnica											
Manual de usuario											
Reporte técnico											
Presentación TT1											
Evaluación de TT1											

Presentación TT2											
Evaluación TT2											

CRONOGRAMA Sánchez García Claudia Emilia TT No:

Título del TT: Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación

Actividad	FEB	MAR	ABR	MAY	JUN	AGO	SEP	OCT	NOV	DIC	ENE
Planificación											
Adquisición de datos											
Preparación de datos											
Entrenamiento de modelos											
Evaluación de modelos											
Despliegue de modelos											
Monitorización de modelos											
Documentación técnica											
Manual de usuario											
Reporte técnico											
Presentación TT1											
Evaluación de TT1											
Presentación TT2											
Evaluación TT2											

7. Referencias

[1] «Semantic Similarity Search for Source Code Plagiarism Detection: An Exploratory Study | Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1», ACM Conferences. Accedido: 12 de septiembre de 2025. [En línea]. Disponible en: https://dl.acm.org/doi/10.1145/3649217.3653622

[2] N. Dunder, S. Lundborg, O. Viberg, y J. Wong, «Kattis vs. ChatGPT: Assessment and Evaluation of Programming Tasks in the Age of Artificial Intelligence», 2 de diciembre de 2023, *arXiv*: arXiv:2312.01109. doi: 10.48550/arXiv.2312.01109.

[2] D. J. P. Villalobo, «La inteligencia artificial y el fraude académico en el contexto universitario», *Rev. Digit. Investig. Postgrado*, vol. 6, pp. 77-93, 2025.

[4] Y. Song, C. Lothritz, X. Tang, T. Bissyandé, y J. Klein, «Revisiting Code Similarity Evaluation with Abstract Syntax Tree Edit Distance», en *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, L.-W. Ku, A. Martins, y V. Srikumar, Eds., Bangkok, Thailand: Association for Computational Linguistics, ago. 2024, pp. 38-46. doi: 10.18653/v1/2024.acl-short.3.

[5] B. Cai, Y. Yu, y Y. Hu, «CSSAM: Code Search via Attention Matching of Code Semantics and Structures», en *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, mar. 2023, pp. 402-413. doi: 10.1109/SANER56733.2023.00045.

[6] A. Neelakantan *et al.*, «Text and Code Embeddings by Contrastive Pre-Training», 24 de enero de 2022, *arXiv*: arXiv:2201.10005. doi: 10.48550/arXiv.2201.10005.

[7] JinXin y LinZhiqiang, «SimLLM: Calculating Semantic Similarity in Code Summaries using a Large Language Model-Based Approach», *Proc. ACM Softw. Eng.*, jul. 2024, doi: 10.1145/3660769.
[8] S. Haque, Z. Eberhart, A. Bansal, y C. McMillan, «Semantic Similarity Metrics for Evaluating Source Code Summarization», 4 de abril de 2022, *arXiv*: arXiv:2204.01632. doi: 10.48550/arXiv.2204.01632.
[9] I. Smit, E. Naudé, y B. Zulu, «A detection process to create awareness of source-code plagiarism among students using it to pass introductory programming», *Indep. J. Teach. Learn.*, vol. 19, n.º 1, pp. 79-92, may 2024, doi: 10.17159/ijtl.v19i1.18854.

8. Alumnos y directo

CARÁCTER: Confidencial
FUNDAMENTO LEGAL: Artículo 11 Fracc. V y Artículos 108, 113 y 117 de la Ley Federal de Transparencia y Acceso a la Información Pública.
PARTES CONFIDENCIALES: Número de boleta y teléfono.

García Rodríguez Erick Daniel. – Alumno de la carrera de Ingeniería en Inteligencia Artificial. Boleta: 2023630222, Tel. 5578666313, email e.danielgrz10@gmail.com.

Firma: _____

Sánchez García Claudia Emilia. – Alumna de la carrera de Ingeniería en Inteligencia Artificial. Boleta: 2021601321, Tel. 5530528517,email emiliasanchezgarcia02@gmail.com

Firma: _____

Manuel Portillo Cedillo.- Maestro en Ciencias en Computación, CINVESTAV-IPN, Ingeniero en Sistemas Computacionales. ESCOM-IPN. Profesor interino de la ESCOM-IPN. Áreas de interés: Inteligencia Artificial, Seguridad Informática y Gestión de proyectos de TI. Tel. 5532262914, email: mportilloc@ipn.mx

Firma: _____

Maribel Aragón García .- Doctor en educación, Maestra en desarrollo de competencias docentes, Maestra en Ciencias con especialidad en Administración de Negocios, Licenciatura en Relaciones Comerciales. Profesora investigadora del Departamento de Formación Integral e Institucional de la ESCOM-IPN. Áreas de interés: Gestión de las tecnologías de información Ext. 52049 email. ipnaragong@gmail.com

Firma: _____